Лабораторная работа №3

Глобальная и кусочная интерполяция

Выполнил: Меркулов Роман Александрович
Группа: 6

Цель работы

Изучение методов глобальной интерполяции (полиномы Лагранжа и Ньютона) и их применение для аппроксимации функции на равномерной и чебышевской сетках, а также интерполяция изображений.

Ход работы

№ этапа	Описание этапа	Результат этапа
1.1	Подготовка. Ознакомление с лабораторной работой, импорт необходимых библиотек (numpy, matplotlib, pandas, PIL). Определение аналитической функции f(x) = -sinc(1.5x).	Составлен план работы, начата работа в Jupyter Notebook.
1.2	Загрузка и отображение исходных данных. Загрузка данных из файла Task_Data_Interpolation.csv и визуализация точечного графика функции на интервале [-5, 5].	Исходные данные успешно загружены и визуализированы.
1.3	Реализация глобальной интерполяции. Созданы функции для построения интерполяционных полиномов: build_lagrange_interpolation и build_newton_interpolation.	Написаны функции, реализующие алгоритмы интерполяции.
1.4	Сравнение методов Лагранжа и Ньютона. Построены графики интерполяции на равномерной сетке из 20 точек и произведено сравнение.	Подтверждена идентичность полиномов Лагранжа и Ньютона. Выявлена катастрофическая погрешность на краях интервала.
1.5	Интерполяция на сетке Чебышева. Создана функция chebyshev_nodes для генерации узлов Чебышева. Проведена интерполяция на этой сетке.	Построены графики, демонстрирующие значительное улучшение точности на краях интервала при использовании узлов Чебышева.

2.1	Загрузка изображения. Загрузка изображения task_vsu_gerb.jpeg и анализ его структуры как трёхмерного массива (RGB).	Изображение загружено, определены его размеры (121×103) и тип данных (uint8), а также в каком виде хранится изображение при его загрузке.
2.2	Реализация методов интерполяции изображений. Разработаны функции для масштабирования изображения: nearest_neighbor_interpolation, linear_interpolation_horizontal, linear_interpolation_vertical, linear_interpolation_both, bilinear_interpolation, bicubic_interpolation.	Написаны функции, реализующие различные алгоритмы интерполяции пикселей.
2.3	Сравнение методов интерполяции изображений. Создана функция compare_interpolation_methods для визуализации и сохранения результатов увеличения изображения в 4 раза всеми методами.	Построены и сохранены изображения для каждого метода. Проведён визуальный анализ их качества.

Выводы

Задание 1:

1. Методы Лагранжа и Ньютона дают идентичные результаты, что подтверждает единственность интерполяционного полинома, однако метод Ньютона удобнее для добавления новых узлов.
2. На равномерной сетке мы наблюдаем, что погрешность на краях интервала достигает десятков и сотен единиц, что делает глобальную интерполяцию на равномерной сетке непригодной.
3. Использование узлов Чебышева кардинально улучшает точность — максимальная погрешность снижается.
4. Выбор узлов интерполяции критически важен — равномерная сетка приводит к катастрофической погрешности, в то время как чебышевская сетка обеспечивает высокую точность на всём интервале.

Задание 2:

1. Ближайший сосед — самый быстрый метод, но приводит к сильной пикселизации и "лестничному эффекту" на наклонных линиях.

2. Билинейная интерполяция (4 соседних пикселя) обеспечивает хороший баланс между скоростью и качеством, эффективно сглаживая пикселизацию.
3. Бикубическая интерполяция (16 соседних пикселей) дает наилучшее качество, создавая более гладкие и чёткие края, но требует больше вычислительных ресурсов.
4. В отличие от этих методов, параметр figsize в matplotlib не выполняет интерполяцию — он просто растягивает изображение при отображении, не добавляя новые пиксели, поэтому его нельзя использовать для реального увеличения изображения.

Импортируемые функции сторонних библиотек

Библиотека	Функция	Назначение	Основные параметры
matplotlib	plt.figure()	Создание нового рисунка	figsize - размер рисунка
matplotlib	plt.plot()	Построение графика	массив значений x, y, color - цвет, linewidth - толщина, label - подпись
matplotlib	plt.title()	Заголовок графика	сам текст, fontsize, fontweight
matplotlib	plt.xlabel()	Подпись оси X	сам текст, fontsize
matplotlib	plt.ylabel()	Подпись оси Y	сам текст, fontsize
matplotlib	plt.grid()	Сетка на графике	Visible – видимость, alpha - прозрачность
matplotlib	plt.legend()	Легенда	fontsize
matplotlib	plt.tight_layout()	Автоматическая подгонка элементов	—
matplotlib	plt.show()	Отображение графика	—
matplotlib	plt.imshow()	Отображение изображения	массив данных изображения
matplotlib	plt.imread()	Чтение изображения в массив NumPy	fname - путь к файлу изображения

numpy	np.linspace()	Создание равномерно распределенных точек	start - начало, stop - конец, num - количество
numpy	np.array()	Преобразование в массив NumPy	object - входные данные
numpy	np.cos()	Косинус	число, массив
numpy	np.sinc()	Функция sinc	число, массив
numpy	np.pi	Число Пи	-
numpy	np.zeros()	Создание массива, заполненного нулями	shape - форма массива; dtype - тип данных
numpy	np.astype()	Преобразование типа данных массива	dtype - целевой тип данных
numpy	np.floor()	Округление вниз	x - число или массив
numpy	np.pad()	Дополнение массива (padding)	array - исходный массив; pad_width - ширина дополнения; mode - режим дополнения
pandas	pd.read_csv()	Чтение CSV-файла	filepath_or_buffer - путь к файлу; sep - разделитель
PIL (Pillow)	Image.fromarray()	Преобразование массива в изображение	массив данных
PIL (Pillow)	Image.save()	Сохранение изображения в файл	fp - путь к файлу
OS	os.path.exists()	Проверка существования пути	path - путь к файлу/директории
OS	os.makedirs()	Создание директории (включая промежуточные)	name - имя директории
Typing	List, Callable	Аннотации типов для статической типизации	List[type] - список элементов типа; Callable[[params], return] - вызываемый объект